

AccountingBench

Authenticated Submission Portal

Final Implementation Guide — Version 3.0

Reflects complete pipeline: task contribution → benchmark run → review → public leaderboard

Auth	Clerk (clerk.com)
Backend	FastAPI — Python
Database	PostgreSQL on Render
File storage	Render disk
Hosting (all)	Render.com
Payments	Stripe (Phase 2)

Accounts needed

1. GitHub — github.com
2. Render — render.com
3. Clerk — clerk.com
4. Stripe — stripe.com (later)

All free to start.

Contents

1. What We Are Building
2. Final Technology Stack
3. Folder Structure
4. Database Schema
5. Adapting the Python Script
6. Batch Runner & Model Re-run Scripts
7. Public Site Data Layer (results.js)
8. Live Public Leaderboard
9. Admin Review Interface
10. Step-by-Step Implementation
11. Security Checklist
12. Effort Estimate
13. All Key Links

1.

What We Are Building

The existing public AccountingBench site remains exactly as it is — no login required. This project adds two things: a private authenticated area where users contribute new benchmark tasks, and a live database-driven public leaderboard that updates automatically when you approve contributions.

1.1 The AccountingBench pipeline — how it actually works

The benchmark is a fixed, growing dataset of accounting tasks. Each task has a question, gold answer, metadata (regulatory framework, category, answer type etc.) and is stored in the PostgreSQL database. The Python pipeline (**pipeline.py**) runs every task against all fixed models, scores the results using the SC/MC formula or LLM-as-a-judge (GPT-5-mini), and writes the scores back to the database.

A user contributes by submitting a NEW task to the dataset. They fill in the task fields on the upload form, upload any supporting documents, and pay. Your Python script then runs that new task through all models automatically. You review the task quality. If approved, it joins the permanent dataset and its scores are averaged into the public leaderboard.

1.2 The complete flow end-to-end

Step	Stage	Description
1	Public site	User visits the public AccountingBench site. Leaderboard shows live aggregate scores from the database.
2	Sign in	User clicks "Sign In". Clerk handles login. Only email addresses from your approved domain list can register.
3	Landing page	User sees their previous submissions and their statuses. Link to submit a new task.
4	Upload form	User fills in the task fields (prompt, answer type, gold answer, regulatory framework, category etc.) and uploads
5	Payment confirmed	Stripe webhook confirms payment. FastAPI saves the task to the database with status = pending and triggers th
6	Script runs	pipeline.py runs the new task through all models (3 trials each), consolidates answers, calls the judge for open-
7	User sees result	The upload page polls every 3 seconds. When the script finishes, the raw model scores appear on the page for
8	You review	In the admin interface, you review the task quality. Approve or reject with one click.
9	Leaderboard updates	If approved, the task is marked is_public = True. The public leaderboard API recalculates aggregate averages a

1.3 What the upload form contains

Field	Description	Required
prompt	The full question text exactly as it will be sent to the models.	Yes
answer_type	Dropdown: single_choice / multi_choice / open_text / open_numeric / journal_entry	Yes
task_type	Dropdown: interpretation_of_law / calculation / journal_entry	Yes
gold_answer	The correct answer. For choice tasks: A, B, C etc. For open tasks: the model answer.	Yes

options	For choice tasks: the answer options as JSON {A: "...", B: "...", C: "..."}.	If choice
regulatory_framework	Dropdown: Austrian Tax Law / IFRS / National GAAP / Mixed etc.	Yes
applicable_regulatory_year	The year the regulation applies.	Yes
category	Dropdown: Tax / Financial Accounting / Management Accounting	Yes
subcategory	Free text subcategory.	Recommended
context	Optional background text provided to the model alongside the question.	No
attached_files	URL or reference to supporting documents.	No
notes	Any additional notes for reviewers.	No
PDF upload	Supporting document (max 20 MB).	No
Excel upload	Supporting spreadsheet (max 20 MB).	No
grading_criteria	For open-text tasks: the rubric used by the LLM judge.	If open
numeric_tolerance	For open_numeric tasks: acceptable +/- tolerance band.	If numeric
education_level	Dropdown: Professional Examinations / University Exams / Secondary Vocational	Yes

2.

Final Technology Stack

What	Service	Cost	Account
Public HTML files	Render Static Site	Free	Render
Private HTML pages	Render Static Site (same)	Free	— same —
Python backend	FastAPI on Render Web Service	Free	— same —
Database	PostgreSQL on Render	Free	— same —
Authentication	Clerk (clerk.com)	Free (up to 10k users)	Clerk
File uploads	Render disk storage	Free	— same —
API keys (models)	OpenAI, Anthropic, Mistral, xAI, DeepSeek, Replicate, OpenAI GPT-5, etc.	Pay per token (varies)	Each provider
Code + autodeploy	GitHub	Free	GitHub
Payments (Phase 2)	Stripe	Free + % per txn	Stripe

The price a user pays per submission must cover your API costs: 3 trials × all models × (input + output tokens) for every task type, PLUS 1 judge call per open-text/numeric/journal-entry task using GPT-5-mini. The settings table in the database stores this price, which you set manually once you have measured the average cost per task.

3.

Folder Structure

```

accountingbench/ ← GitHub repository root
■
■■■■ .env ← all secrets – never commit to GitHub
■■■■ CLAUDE.md ← Claude Code instructions for this repo
■■■■ reset_db.py ← utility: wipe all tasks/submissions from DB
■
■■■■ public/ ← existing public site (static HTML/CSS/JS)
■ ■■■■ index.html ← overview page (fully dynamic)
■ ■■■■ leaderboard.html
■ ■■■■ dashboard.html ← analytics dashboard (fully dynamic)
■ ■■■■ methodology.html
■ ■■■■ about.html ← 5 team members
■ ■■■■ styles.css ← all CSS centralised here
■ ■■■■ main.js
■ ■■■■ results.js ← single source of truth for ALL benchmark data
■ ■■■■ data.js ← derived data + DOM render functions
■
■■■■ auth-pages/ ← private authenticated HTML pages
■ ■■■■ sign-in.html
■ ■■■■ register.html
■ ■■■■ landing.html ← submission history + new submission link
■ ■■■■ upload.html ← task contribution form + result display
■
■■■■ backend/ ← FastAPI Python server
■■■■ main.py ← app entry point, CORS, route registration
■■■■ auth.py ← Clerk JWT verification dependency
■■■■ submissions.py ← upload form endpoint, file handling
■■■■ admin.py ← approve/reject endpoint, domain mgmt
■■■■ leaderboard.py ← public aggregate scores API endpoint
■■■■ payments.py ← Stripe webhook handler (Phase 2)
■■■■ models.py ← SQLAlchemy database table definitions
■■■■ database.py ← DB engine and session factory
■■■■ batch_run.py ← batch import from Excel + run all models
■■■■ rerun_model.py ← run new model(s) on existing DB tasks
■■■■ batch_utils.py ← shared helpers for batch scripts
■■■■ processing/
■ ■■■■ pipeline.py ← real benchmark pipeline (parallel models)
■■■■ uploads/ ← uploaded PDF and Excel files
■■■■ requirements.txt

```

4.

Database Schema

Seven tables cover the complete system. All are in PostgreSQL (SQLite locally for development). Your Python script connects to this database directly via SQLAlchemy.

4.1 users

id	String — the Clerk user ID (e.g. user_2abc...). Primary key.
email	String, unique. Stored for display and admin.
first_name	String.
last_name	String.
created_at	DateTime.
is_active	Boolean, default True. Set False to block a user.

4.2 allowed_domains

id	Integer, primary key.
domain	String, unique. Example: kpmg.com
added_at	DateTime.
added_by	String — your admin email.

4.3 benchmark_tasks

This table replaces the ground-truth Excel spreadsheet. The 521 existing tasks are imported here. Every new user-contributed task is added here too.

id	Integer, primary key, auto-increment.
question_id	String, unique. E.g. "12008933_0001". Auto-generated on insert.
prompt	Text. The full question.
answer_type	String. single_choice / multi_choice / open_text / open_numeric / journal_entry
task_type	String. interpretation_of_law / calculation / journal_entry
task_format	String. Optional.
gold_answer	Text. The correct answer.
options	JSON. Answer options for choice tasks.
grading_criteria	Text. Rubric for LLM judge on open tasks.
numeric_tolerance	Float, nullable. Tolerance band for open_numeric tasks.
acceptable_variants	Text, nullable. Alternative valid answers.
context	Text, nullable.
attached_files	Text, nullable. File references.

regulatory_framework	String.
applicable_regulatory_year	Integer.
category	String. Tax / Financial Accounting / Management Accounting
subcategory	String, nullable.
skills	String, nullable.
education_level	String. Professional Examinations / University Exams / Secondary Vocational
notes	Text, nullable.
source	String. "user_submitted" or "original_dataset"
submitted_by	String, nullable. FK → users.id. Null for original 521 tasks.
is_public	Boolean, default False. Set True after you approve a user submission.
created_at	DateTime.
validated_by	String, nullable. Your admin email after review.
validation_status	String. pending / approved / rejected
pdf_path	String, nullable. Path to uploaded PDF.
excel_path	String, nullable. Path to uploaded Excel.

4.4 benchmark_runs

One row per model per task submission run. Mirrors the Runs sheet from the existing Excel pipeline.

id	Integer, primary key.
run_id	String. UUID. E.g. "run_20260415_143022_a1b2c3d4"
task_id	Integer, FK → benchmark_tasks.id
model_name	String. E.g. "gpt-5.4"
run_timestamp	DateTime.
temperature	Float. Always 0.0.
n_trials	Integer. Always 3.
system_prompt_version	String. E.g. "v3_types"
dataset_version	String. E.g. "v3"
judge_model	String. Always "gpt-5-mini"
inference_notes	Text. Any additional run metadata.

4.5 benchmark_outputs

One row per model per task. Stores all trial answers, scores, and judge output.

id	Integer, primary key.
run_id	String, FK → benchmark_runs.run_id
task_id	Integer, FK → benchmark_tasks.id
model_name	String.
model_answer_1	Text. Trial 1 answer.
model_confidence_1	Float. Trial 1 confidence.
model_answer_2	Text. Trial 2 answer.
model_confidence_2	Float.
model_answer_3	Text. Trial 3 answer.
model_confidence_3	Float.
final_answer	Text. Consolidated answer (majority vote or judge consolidation).
avg_model_confidence	Float.
score_percent_sc_mc	Float, nullable. SC/MC formula score (choice tasks only).
judge_score_percent	Float, nullable. LLM-as-a-judge score (open tasks only).
judge_confidence	Float, nullable.
final_score_percent	Float. The score that counts for the leaderboard.
evaluation_method	String. sc_mc_formula or judge.
token_input	Integer.
token_output	Integer.
token_reasoning	Integer, nullable.
evaluated_at_utc	DateTime.
evaluation_notes	Text.

4.6 submissions

Tracks each user's contribution from form submission through payment, script execution, and review.

id	Integer, primary key.
user_id	String, FK → users.id
task_id	Integer, FK → benchmark_tasks.id. Set after task is inserted.
status	String. pending → processing → done → error
payment_status	String. unpaid → paid → refunded. (Phase 2)
stripe_payment_id	String, nullable. Stripe Payment Intent ID. (Phase 2)
price_charged	Integer, nullable. Amount in cents. (Phase 2)
submitted_at	DateTime.
completed_at	DateTime, nullable. When the script finishes.

4.7 settings

Single-row configuration table. Change the price here without touching code.

id	Integer. Always 1.
price_per_submission	Integer. Cents. E.g. 5000 = €50.00. Set after measuring average API cost. (Phase 2)
currency	String. E.g. eur. (Phase 2)
updated_at	DateTime.

5.

Adapting the Python Script for the Database

The existing benchmark script reads from Excel files and writes to Excel files. It has been adapted to read tasks from the database and write results back to the database. The core logic — the three-trial inference loop, SC/MC scoring, judge calls, consolidation — stays completely unchanged.

5.1 What changes in the script

Currently (Excel)	Adapted (Database)
<code>pd.read_excel(GROUND_TRUTH_XLSX)</code> – reads all tasks	<code>db.query(BenchmarkTask).filter_by(id=task_id)</code> – reads one specific task
<code>pd.read_excel(MODEL_CALL_XLSX)</code> – reads model call templates	task_id is passed directly as a Python dict – no Excel needed
<code>append_row_respecting_formulas(ws_runs, ...)</code> – writes to Excel	<code>db.add(BenchmarkRun(...)) + db.commit()</code> – writes to PostgreSQL
<code>append_row_respecting_formulas(ws_out, ...)</code> – writes to Excel	<code>db.add(BenchmarkOutput(...)) + db.commit()</code> – writes to PostgreSQL
<code>wb.save(EVALUATION_XLSX)</code> – saves the workbook	Changes are committed row by row as the script runs
Script is run from command line manually	Script is triggered as a FastAPI background task after form submit

5.2 The pipeline wrapper (backend/processing/pipeline.py)

Rather than modifying the script heavily, wrap it. The wrapper handles the database connection and calls the adapted script functions. Models run in parallel via `ThreadPoolExecutor` — each thread gets its own DB session. Per-model timeout is configured via the `MODEL_REGISTRY_JSON` "timeout" key.

```
from backend.database import SessionLocal
from backend.models import BenchmarkTask, BenchmarkRun, BenchmarkOutput, Submission
from concurrent.futures import ThreadPoolExecutor, as_completed

def run_pipeline(submission_id: int):
    db = SessionLocal()
    try:
        submission = db.query(Submission).filter_by(id=submission_id).first()
        submission.status = "processing"
        db.commit()
        task = db.query(BenchmarkTask).filter_by(id=submission.task_id).first()
        # Run all models in parallel
        with ThreadPoolExecutor(max_workers=len(ALL_MODELS)) as executor:
            futures = {executor.submit(run_single_model, task, m, submission_id): m
                       for m in ALL_MODELS}
        for future in as_completed(futures):
            future.result() # raises on error
        submission.status = "done"
        db.commit()
    except Exception as e:
        submission.status = "error"
        db.commit()
    finally:
        db.close()
```

5.3 ALL_MODELS list — 13 models as of March 2026

```
ALL_MODELS = [  
  "gpt-5.5", # OpenAI  
  "gpt-5.4", # OpenAI  
  "claude-opus-4-7", # Anthropic  
  "gpt-5.2", # OpenAI  
  "claude-opus-4-6", # Anthropic  
  "claude-sonnet-4-6", # Anthropic  
  "gpt-5-mini", # OpenAI  
  "Kimi-K2.6", # Moonshot AI  
  "Mistral-Large-3", # Mistral AI  
  "grok-4-fast-reasoning", # xAI  
  "gpt-4o", # OpenAI  
  "DeepSeek-V3.2", # DeepSeek  
  "mercury-2", # Inception Labs  
]
```

5.4 API keys and model registry in .env

All model provider API keys and model configurations live in the .env file at the project root. OPENAI_API_KEY must be set (can be "not-used") even if all models use MODEL_REGISTRY_JSON.

```
OPENAI_API_KEY=not-used # Required even if all models use MODEL_REGISTRY_JSON  
MODEL_REGISTRY_JSON={"gpt-5.4": {"api_type": "openai_v1", "base_url": "...", "api_key": "..."}, ...}  
# Add entries for Anthropic, Mistral, xAI, DeepSeek, Moonshot AI, Inception Labs  
# Never commit .env to GitHub
```

6.

Batch Runner & Model Re-run Scripts

Three scripts handle bulk operations outside the web submission flow. All live in backend/ and are run from the project root.

6.1 batch_run.py — import from Excel and run all models

Reads tasks from an Excel file, upserts them into benchmark_tasks, and runs the full pipeline on all models in parallel. Use this for the initial bulk import of the 521 original tasks.

```
# Dry run — show what would happen without writing anything
python -m backend.batch_run --file backend/tasks.xlsx --approved --dry-run
# Full import and run (4 workers for 13 models)
python -m backend.batch_run --file backend/tasks.xlsx --approved --max-workers 4
# Skip tasks already in DB (safe to re-run)
python -m backend.batch_run --file backend/tasks.xlsx --approved --skip-existing
```

6.2 rerun_model.py — run new model(s) on existing DB tasks

Runs one or more NEW models against ALL tasks already in the database. No Excel file needed. Existing benchmark_outputs for the requested model(s) are skipped — safe to run multiple times.

```
# Step 1: Always dry-run first — no API calls, no DB writes
python -m backend.rerun_model --models "Kimi-K2.6" --dry-run
# Step 2: Test on exactly 1 task
python -m backend.rerun_model --models "Kimi-K2.6" --limit 1 --sequential
# Step 3: Full run (15 workers for 1 model)
python -m backend.rerun_model --models "Kimi-K2.6" --max-workers 15
# Multiple models at once
python -m backend.rerun_model --models "Kimi-K2.6,gpt-5.5" --max-workers 4
```

6.3 Recommended max-workers by model count

Models being re-run	Recommended --max-workers	Connections used
1	15	15
3	15	45
13	4	52

6.4 batch_utils.py — shared helpers

Both batch_run.py and rerun_model.py import shared helpers from batch_utils.py. Do not add script-specific logic here.

```
from backend.batch_utils import (
    ensure_batch_user, # create batch user in DB if missing
    create_submission, # create a pending Submission row
```

```
get_existing_model_outputs, # check which models already have outputs
log_summary, # standardised run summary
BATCH_USER_ID, # default user for batch submissions
)
```

7.

Public Site Data Layer (results.js)

The public site is fully dynamic — all numbers, charts, tables, and leaderboards are driven from two files. Never edit numbers directly in HTML files.

7.1 results.js — single source of truth

Contains BENCHMARK_RESULTS (one object per model) and BENCHMARK_META (dataset composition, task type counts, education level distribution etc.). To add a new model or update scores, edit only this file.

Key fields per model:

Field	Description
overall, tax, financial, management	Category scores (%)
interplaw, calculation, journal	Task type scores (%)
multiChoice, openText, singleChoice, answerTypeScores	Answer type scores (%)
austrianTax, mixedAcc, ugb, ifrs	Regulatory framework scores (%)
eduProf, eduMaster, eduVoc	Education level overall scores (%)
byEdu: { prof, master, voc }	Full score breakdown per education level for all subcategories. null = no tasks in that combination (
n	Tasks completed, e.g. "520" or "465/520"
priceIn, priceOut	Token prices in USD per 1M tokens
cost	Pre-calculated avg cost per task in USD
tokTask, speed	Avg tokens per task, output tokens/sec
calib	Confidence calibration points [{x, y, n}]
note	Optional warning shown in leaderboard

7.2 data.js — render functions

Contains all DOM render functions. Called once on page load to build the entire UI from BENCHMARK_RESULTS. The holistic matrix accepts an eduKey parameter ("all", "prof", "master", "voc") to filter by education level.

Function	What it renders
renderTicker()	Scrolling score bar at the top
renderOverviewLeaderboard()	Desktop leaderboard table (sorted by overall)
renderHolisticMatrix(eduKey)	Full results matrix with grouped headers and education filter
renderCostTable()	API Cost Analysis table
renderTokenBars()	Avg. Token Usage bar chart
renderQuestionDistribution()	Donut chart + legend

renderDatasetTables()	All dataset composition tables
-----------------------	--------------------------------

7.3 Education-level filter on the holistic matrix

The overview page has a filter bar above the holistic matrix with four buttons: All / Professional Exams / University Exams / Secondary Vocational. Clicking a filter re-renders the matrix using byEdu data from results.js.

Key insight: IFRS shows — (no tasks) when "Professional Exams" is selected, because all 25 IFRS tasks are exclusively Master/University level. This makes the near-ceiling IFRS score story immediately visible to the reader.

8.

Live Public Leaderboard

Currently leaderboard numbers are hardcoded in results.js. After deployment, the public site can optionally fetch live aggregate scores from the database via a single API endpoint. Only approved tasks (`is_public = True`) are included in the calculation.

8.1 The leaderboard API endpoint (backend/leaderboard.py)

```
@app.get("/api/leaderboard")
async def get_leaderboard():
    """
    Returns average scores per model across all is_public=True tasks.
    Groups by model_name, averages final_score_percent.
    Also breaks down by category (Tax, Fin. Acc., Mgmt. Acc.)
    """
    db = SessionLocal()
    approved = db.query(BenchmarkTask.id).filter_by(is_public=True).all()
    approved_ids = [t.id for t in approved]
    outputs = db.query(BenchmarkOutput).filter(
        BenchmarkOutput.task_id.in_(approved_ids)
    ).all()
    # Aggregate per model ...
    return sorted(result, key=lambda x: -x["overall"])
```

8.2 Updating the public HTML pages

In the public HTML pages, replace the hardcoded `BENCHMARK_RESULTS` in results.js with a `fetch()` call when deploying the live leaderboard:

```
// In main.js – replace hardcoded data with live fetch
async function loadLeaderboard() {
  const res = await fetch("https://your-api.onrender.com/api/leaderboard");
  const data = await res.json();
  // data matches the BENCHMARK_RESULTS array format
  renderOverviewLeaderboard(data);
}
window.addEventListener("load", loadLeaderboard);
```

8.3 Caching the leaderboard response

The leaderboard does not change often — only when you approve a submission. Add a simple 5-minute cache to avoid hitting the database on every public page load:

```
from fastapi_cache.decorator import cache
@app.get("/api/leaderboard")
@cache(expire=300) # cache for 5 minutes
async def get_leaderboard():
    ...
```

9.

Admin Review Interface

After the Python script finishes, you see the new task in a simple admin page. You review the task text for quality and click Approve or Reject. No code-level access needed for the day-to-day review.

9.1 Admin page (auth-pages/admin.html)

A protected HTML page only accessible with your admin email (checked by Clerk + FastAPI). Shows all tasks with status = pending:

- Task prompt, answer type, category, regulatory framework, gold answer.
- Uploaded documents as download links.
- The model scores from the benchmark run.
- Approve and Reject buttons.

9.2 Approve / Reject endpoints (backend/admin.py)

```
@app.post("/admin/tasks/{task_id}/approve")
async def approve_task(task_id: int, user=Depends(require_admin)):
    db = SessionLocal()
    task = db.query(BenchmarkTask).filter_by(id=task_id).first()
    task.is_public = True
    task.validation_status = "approved"
    task.validated_by = user
    db.commit()
    return {"ok": True}

@app.post("/admin/tasks/{task_id}/reject")
async def reject_task(task_id: int, user=Depends(require_admin)):
    db = SessionLocal()
    task = db.query(BenchmarkTask).filter_by(id=task_id).first()
    task.is_public = False
    task.validation_status = "rejected"
    task.validated_by = user
    db.commit()
    return {"ok": True}
```

9.3 What the user sees after rejection

Nothing is shown to the user about the review decision. On their landing page, the submission appears with status "rejected" and no score is shown. The task is not included in any public numbers.

10.

Step-by-Step Implementation

Phase 1 — Project Setup + Database

Estimated time: ~1.5 hours

- 1. Create the folder structure from Section 3 and push to GitHub.
- 2. Create a PostgreSQL database on Render (New → PostgreSQL, free tier).
- 3. Set up the Python virtual environment and install dependencies.
- 4. Write models.py with all seven tables from Section 4.
- 5. Run Alembic to create the tables: alembic init migrations → alembic revision --autogenerate → alembic upgrade head
- 6. Import the 521 existing tasks using batch_run.py: python -m backend.batch_run --file backend/tasks.xlsx --approved
- 7. Verify in TablePlus or DBeaver: 521 rows in benchmark_tasks, all with is_public=True.
- **Test: the 521 existing tasks are in the database.**

Phase 2 — Authentication (Clerk)

Estimated time: ~1 hour

- 8. Create a Clerk application. Under Restrictions → Allowlist, enable domain-based registration.
- 9. Add allowed domains (e.g. kpmg.com) in the Clerk dashboard.
- 10. Add the Clerk publishable key and secret key to .env.
- 11. Write backend/auth.py — the get_current_user dependency that verifies the Clerk JWT.
- 12. Write backend/auth.py — a require_admin dependency that checks the user email matches ADMIN_EMAIL from .env.
- 13. Add the "Sign In" link to the public nav bar in each HTML page.
- 14. Mount the Clerk sign-in and register components in sign-in.html and register.html.
- **Test: register with an allowed email domain. Verify sign-in works. Try a non-allowed domain — confirm it is blocked.**

Phase 3 — Upload Form + Python Script Integration

Estimated time: ~3 hours

- 3a Upload HTML page: build the form with all fields from Section 1.3.
- 3b Submission endpoint: create POST /submissions/prepare with auth, validation, file saving, and pipeline trigger.
- 3c Python script: ensure pipeline.py reads from DB and writes results back to benchmark_runs and benchmark_outputs.
- 3d Status polling: poll GET /submissions/{id}/status every 3 seconds and display scores when done.
- **Test: submit a new task. Confirm 13 run rows and 13 output rows appear in the DB. Status changes to "done".**

Phase 4 — Landing Page + Admin Interface

Estimated time: ~1.5 hours

- 27. Protect landing.html with a Clerk session check.
- 28. Fetch and display submission history from GET /submissions/mine.
- 29. Add a "Contribute New Task" button linking to upload.html.
- 30. Build admin.html — protect with require_admin, show pending tasks with Approve/Reject buttons.
- 31. Add a domain management section: list/add/delete allowed domains.

Phase 5 — Live Leaderboard + Public Site Update

Estimated time: ~1 hour

- 35. Write the GET /api/leaderboard endpoint from Section 8.1.
- 36. Add fastapi-cache2 and wrap with @cache(expire=300).
- 37. Optionally update main.js to replace hardcoded data with a fetch() call.
- 38. Test: approve a task, reload the leaderboard, confirm numbers update.

Phase 6 — Deployment to Render

Estimated time: ~1 hour

- 40. Push everything to GitHub.
- 41. On Render: New → Static Site → publish directory: public → deploy.
- 42. On Render: New → Static Site → publish directory: auth-pages → deploy.
- 43. On Render: New → Web Service → root directory: backend → start command: uvicorn main:app --host 0.0.0.0 --port 10000
- 44. In Render Web Service → Environment Variables, add every .env variable.
- 45. Update DATABASE_URL to the Render PostgreSQL external URL.
- 46. In main.py, add your Render Static Site URLs to CORS allowed origins.
- 47. In Clerk dashboard → Domains, add both Render Static Site URLs.
- 48. Update all fetch() URLs from localhost:8000 to the Render Web Service URL.
- 49. Run alembic upgrade head against the production database.
- 50. Run batch_run.py to import tasks into production.

Phase 7 — Payments with Stripe (Phase 2 — build later)

Design is in place — add when ready. The database already has payment_status, stripe_payment_id, price_charged in submissions, and the settings table. No schema changes needed.

- 51. Measure average API cost per task from token_input + token_output. Set price_per_submission in settings.
- 52. pip install stripe.
- 53. On form submit, create a Stripe Payment Intent and redirect to Stripe Checkout.
- 54. Add POST /payments/webhook. Verify Stripe-Signature. On payment_intent.succeeded, trigger run_pipeline().

Reference: stripe.com/docs/checkout/quickstart

■ **Render sleep warning:** Render's free Web Service sleeps after 15 minutes of inactivity. The first request after sleep takes ~30 seconds to wake. The \$7/month paid tier removes this.

11.

Security Checklist

Auth	Clerk handles all passwords, tokens, and email verification. You never store or see passwords.
JWT verification	Every protected FastAPI endpoint calls <code>get_current_user</code> which verifies the Clerk token. Never trust user ID.
Admin endpoints	<code>/admin/*</code> routes call <code>require_admin</code> which checks the verified user email against <code>ADMIN_EMAIL</code> in <code>.env</code> .
API keys	All model provider API keys in <code>.env</code> only. Never in code. Never in GitHub. Added via Render Environment Variables.
File validation	Check extension AND MIME type for uploads. Reject anything not PDF or Excel. Set 20 MB max file size.
SQL injection	SQLAlchemy ORM prevents this by default. Never build raw SQL strings with user input.
CORS	Only list your Render Static Site URLs in <code>allow_origins</code> . Never use <code>allow_origins=["*"]</code> in production.
HTTPS	Render serves HTTPS on all services automatically.
Stripe webhooks	Always verify the Stripe-Signature header before processing any payment event. (Phase 2)
Submission access	<code>/submissions/mine</code> filters by the verified user ID. Users cannot see each other's tasks or scores.
Leaderboard data	GET <code>/api/leaderboard</code> filters by <code>is_public=True</code> . Pending and rejected tasks are never exposed publicly.

12.

Effort Estimate

Phase	Task	Time
1	Setup, database, import 521 tasks	1.5 hrs
2	Clerk auth, domain restriction, sign-in pages	1 hr
3	Upload form, submission endpoint, Python script adaptation, polling	3 hrs
4	Landing page, admin review interface	1.5 hrs
5	Live leaderboard API, update public site	1 hr
6	Deployment to Render	1 hr
—	Total Phase 1 (without payments)	~9 hours
7	Stripe pay-per-submission (Phase 2)	2–3 hrs

Build and test locally through Phase 5 before deploying. FastAPI's automatic docs at <http://localhost:8000/docs> let you test every endpoint without any frontend. Run the Python script locally first against a test task using `--limit 1 --sequential` to confirm it connects to the database correctly — this is the most complex integration point.

13.

All Key Links

Accounts	
GitHub	github.com — free
Render	render.com — free tier, no credit card
Clerk	clerk.com — free up to 10,000 monthly active users
Stripe	stripe.com — free account; pay per transaction when live (Phase 2)
OpenAI	platform.openai.com — for GPT models and the judge (GPT-5-mini)
Anthropic	console.anthropic.com — for Claude models
Mistral	console.mistral.ai — for Mistral-Large-3
xAI	console.x.ai — for grok-4-fast-reasoning
DeepSeek	platform.deepseek.com — for DeepSeek-V3.2
Moonshot AI	platform.moonshot.cn — for Kimi-K2.6
Inception Labs	inceptionlabs.ai — for mercury-2
Documentation	
FastAPI — getting started	fastapi.tiangolo.com/tutorial/
FastAPI — background tasks	fastapi.tiangolo.com/tutorial/background-tasks/
FastAPI — file uploads	fastapi.tiangolo.com/tutorial/request-files/
FastAPI — CORS	fastapi.tiangolo.com/tutorial/cors/
SQLAlchemy — ORM quickstart	docs.sqlalchemy.org/en/20/orm/quickstart.html
Alembic — migration tutorial	alembic.sqlalchemy.org/en/latest/tutorial.html
fastapi-cache2	github.com/long2ice/fastapi-cache
Clerk — JavaScript quickstart	clerk.com/docs/quickstarts/javascript
Clerk — FastAPI integration	clerk.com/docs/backend-requests/overview
Clerk — Allowlist	clerk.com/docs/authentication/allowlist
Render — deploy FastAPI	render.com/docs/deploy-fastapi
Render — PostgreSQL	render.com/docs/postgresql-creating-connecting
Stripe — Checkout quickstart	stripe.com/docs/checkout/quickstart
Stripe — Webhooks	stripe.com/docs/webhooks
TablePlus (free DB viewer)	tableplus.com

DBeaver (free DB viewer)	dbeaver.io
--------------------------	------------